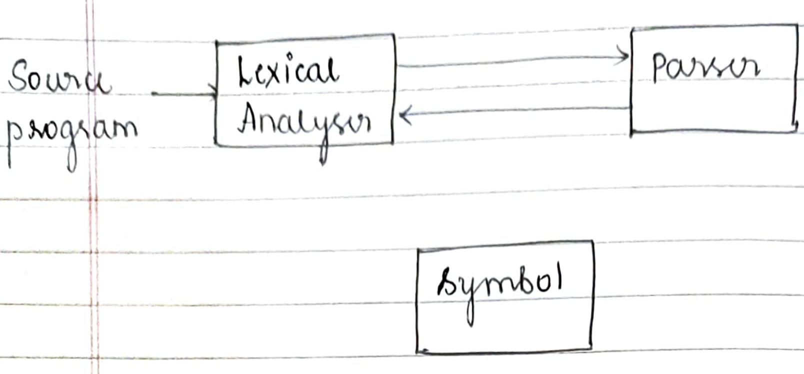
System compilers.

Chapter 3: Lexical Analysis.

⇒ Role :



⇒ Attributes : <tokenname, attribute value>.

1. E = M * C ** 2

<id, pointer to symbol-table entry for E>
<assign-op>
<id, pointer to symbol-table entry for M>
<mult-op>
<id, pointer to symbol-table entry for C>
<exp-op>
<number, integer value 2>

Add	Lexeme	Type	line.no
1000	E	float	2
2000	M	float	2
3000	C	float	2

<id, 1000>
<assign>
<id, 2000>
<mult>
<id, 3000>
<exp-op>
<number, 2>

2. $si = \frac{p * t * r}{100}$

Add	Lexeme	Type	line no
1001	si	float	2
1002	p	float	2
1003	t	float	2
1004	r	float	2

<id, 1001>
<assign-op>
<id, 1002>
<mult-op>
<id, 1003>
<mult-op>
<id, 1004>
<div-op>
<number, ~~integer value~~ 100>

⇒ Lexical errors :
Recovery strategy - panic mode .

Imp ⇒ Input buffering : The method of reading a block of characters (1k or 4k or more bytes) from the disk in one read operation and storing in memory (normally in the form of array) for futher processing and faster accessing is called input buffering.

Lexemebegin - will point to the current lexeme.
forward - scans the pattern futher.

⇒ Strings and languages :
Alphabet is a finite set of symbols.
eg : symbols are letters, digits & punctuations.

classmate
Date
Page

String : It is a finite sequence of symbols drawn from the alphabet.
* Length of string s is |s|, is the number of occurrences of symbols in s.
*. Empty string : ϵ.

Language : A language is any countable set of strings over some fixed alphabet.

Operations on languages :
Union of L and M :

$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$.

Concatenation of L and M :

$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$.

Kleen closure of L :

$L^* = \bigcup_{i=0}^{\infty} L^i$

Positive closure of L :

$L^+ = \bigcup_{i=1}^{\infty} L^i$.

⇒ Regular expressions :
It describes the set of strings in particular language.

Union : $(r)|(s) = L(r) \cup L(s)$
Concatenation : $(r)(s) = L(r)L(s)$

$(r)^* = (L(r))^*$

$r = L(r)$.

. '' has highest precedence and has left associativity.
*. Concatenation has second highest precedence

'?'

and has left associativity.

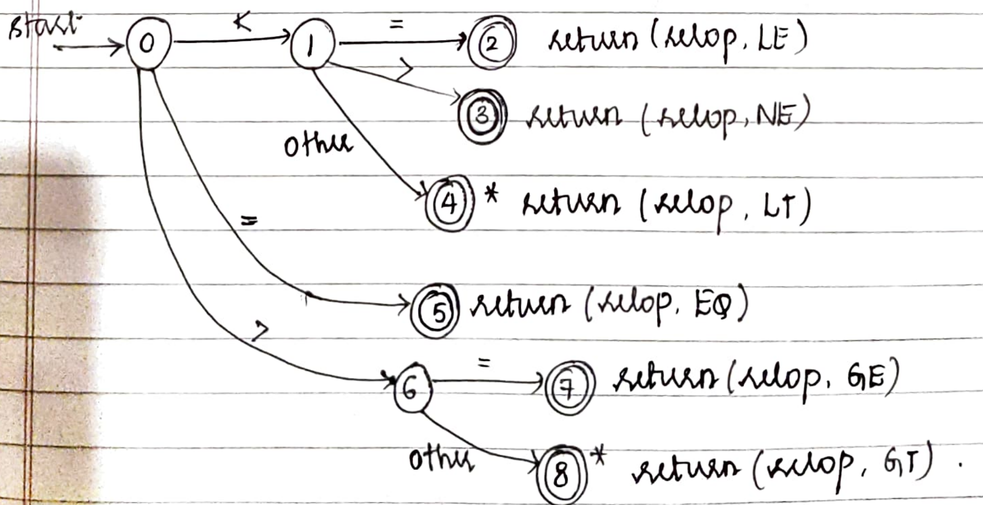
*. '|' has lowest precedence.

1. Digit → [0-9]   letter → [A-Za-z].
2. digits → [0-9]* / [digit]*.
3. number → (digit)(.)(digits)*   (3.124).
4. '?' → 0 or more occurrences.
6. Exponent → (digit)(.)(digit)? E (digit)*   (3.124E^{234}).
   (digit)(.)(digit)?(E [+-]? (digit)*).

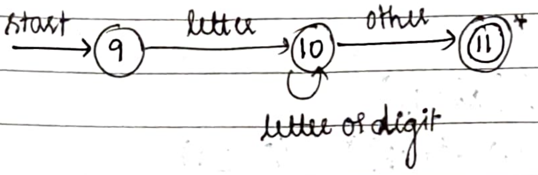
⇒ Extensions of regular expressions

1. One or more instances : '+'.
2. Zero or one instance : '?' has same associativity & precedence as '+' & '*'.
3. Character class

Lexeme	Token-value	Attribute value
<	relop	LT
<=	"	LE
=	"	EQ
<>	"	NE
>	"	GT
>=	"	GE.

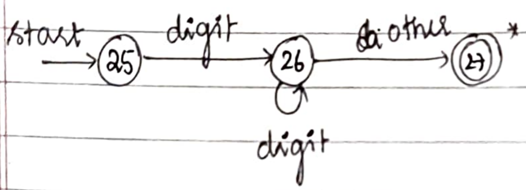


2. Transition diagram for ~~letters~~ identifiers and keywords



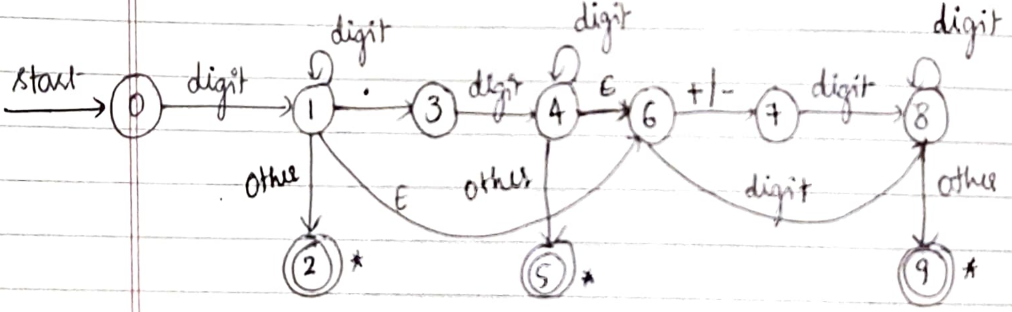
return (gettoken(), install_id).

3. Transition diagram for unsigned integer numeric constant.



install_num()

4. Transition diagram for unsigned numbers.



5. Transition diagram for whitespace (new line tab space and space)